COFFEESHOP PROJECT / LEARNING GUIDE

Building an Infinite Moving Carousel with Free Scrolling

How My CoffeeShop Carousel Works - From Beginner to Builder

The promise of this guide

By the end, you should be able to rebuild this kind of carousel yourself. We will focus on the reasons behind each decision, not just the lines of code.

Native scrolling • duplicated card sets • pause and resume • requestAnimationFrame
Prepared from the current CoffeeShop implementation

Table of Contents

> **Project note**
> In the current project, the cards are written in `market-page.html`. The first 15 cards appear again immediately after them in the same `.coffee-menue` track. JavaScript does not generate the duplicate set; it measures and moves the existing DOM.

Files involved

File	Job
`market-page.html`	Contains the one scroll container, one horizontal track, and the original plus duplicate card markup.
`style.css`	Makes the container scroll horizontally and the track wide, horizontal, and non-shrinking.
`Market-Folder/Market.js`	Moves `scrollLeft`, finds the loop point, pauses for interaction, and keeps heart/card behavior running.
`Market-Folder/Market-Elements.js`	Finds the menu hearts with `.MenueCoffee-Heart`; the footer heart is not selected.

CHAPTER 1

The problem we were trying to solve

We wanted one coffee row to move by itself, remain normally scrollable, stop when a person takes control, restart after the interaction, and repeat forever without showing a blank patch.

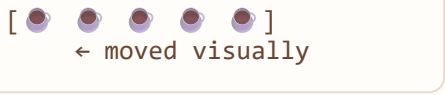
The difficult part is that automatic movement and human movement both need to control the same horizontal position.

Core decision

Use the browser's real scroll position, `scrollLeft`, as the single source of movement. Do not move the carousel with a CSS transform.

Transform movement vs real scrolling

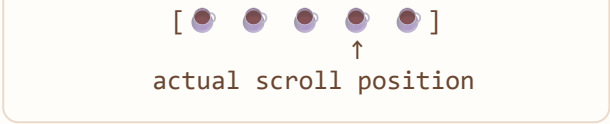
Transform



`transform: translateX(...)` changes where an element is painted. The scroll box does not think its scroll position changed.

If a user tries to scroll at the same time, two systems can fight: CSS moves the track while the browser is trying to move the scroll position.

scrollLeft



`container.scrollLeft` changes the native horizontal position of the scroll box.

That is the same position the user controls with a mouse, trackpad, scrollbar, or finger. One control path is easier to pause and resume.

Why the old percentage was fragile

`translateX(-50%)` is a visual percentage. It does not know the exact width of one coffee set, including responsive card widths and the 20px gap. A transform can therefore expose empty space or reset at the wrong visual moment.

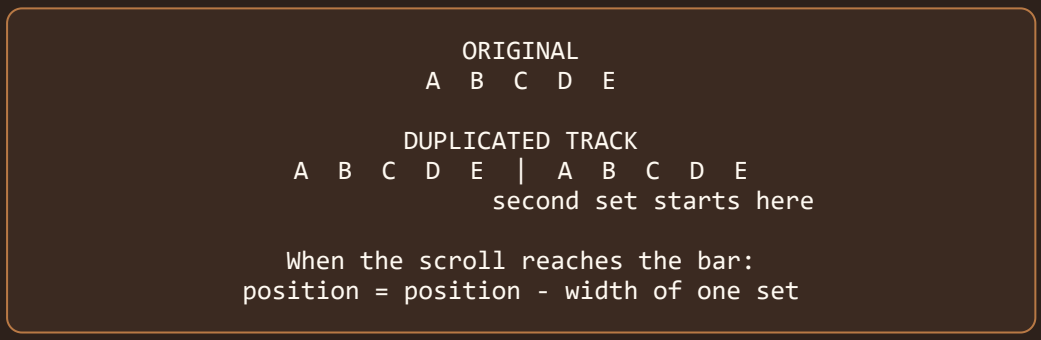
Design rule

Let native scrolling do the movement. Make two identical sets. When the second set begins, subtract the measured width of one complete set.

CHAPTER 2

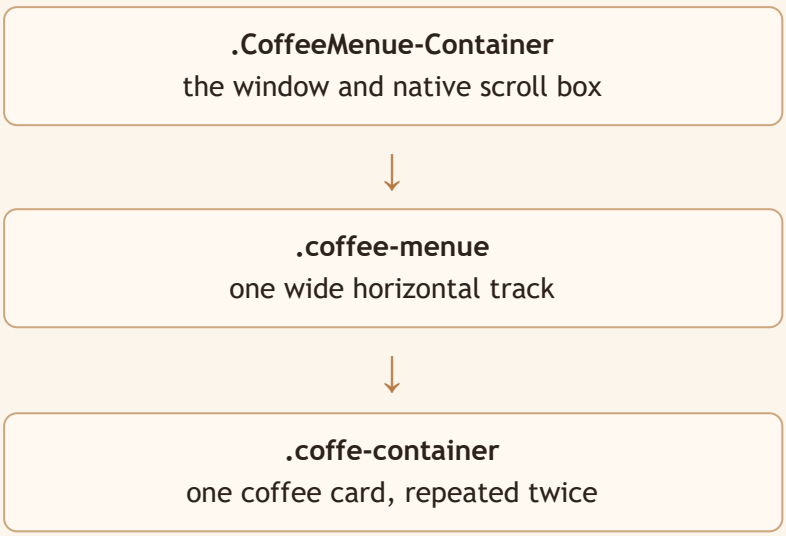
The duplicate-set trick

An infinite carousel is not truly infinite. It is two identical sets pretending to be one long conveyor belt.



After the subtraction, the viewer sees the same card position in the first set. Because the cards on both sides are identical, the jump is hidden.

Understand the structure




```
<div class="CoffeeMenue-Container">
  <div class="coffee-menue">
    <div class="coffe-container">...</div>
    <!-- the first 15 cards repeat here -->
  </div>
</div>
```

Rule	Reason
`overflow-x: auto`	Keep real horizontal scrolling available. The browser can provide mouse, trackpad, scrollbar, and touch scrolling.
`display: flex`	Put cards in one row.
`width: max-content`	Let the track be as wide as its cards require instead of forcing it into the viewport.
`flex-shrink: 0`	Prevent cards or the track from shrinking and changing the measured loop distance.
`scrollbar-width: none`	In this project, the visible scrollbar is hidden while native scrolling still works.

Common mistake

`overflow-x: hidden` would hide the very native scrolling this design depends on.

Variables: the small control room

Variable	Stores	Why it exists / analogy
`CoffeeMenueContainer`	The outer scroll element	Where the browser keeps `scrollLeft`. It is the window you move.
`CoffeeMenue`	The inner track	The long conveyor belt holding every card.
`CoffeeCards`	All direct card children	The inventory list. We need positions of the first and duplicate sets.
`CoffeeSetSize`	Half the card count	How many cards belong to one set. With 30 cards, it is 15.
`CoffeeMenuePaused`	A boolean	The STOP button: `true` means automatic movement waits.
`CoffeeMenueResumeTimer`	A timeout handle	An alarm scheduled to turn movement back on.
`CoffeeMenueLastTime`	The previous frame timestamp	The last clock reading, used to measure elapsed time.
`CoffeeMenueWritingScroll`	A boolean guard	A sign saying “JavaScript caused this scroll; do not call it user input.”

Selecting the elements

```
const CoffeeMenueContainer = document.querySelector(".CoffeeMenue-Container");
const CoffeeMenue = document.querySelector(".coffee-menue");
const CoffeeCards = CoffeeMenue.querySelectorAll(":scope > .coffe-container");
```

`querySelector` returns the first matching element. `querySelectorAll` returns all matches. The selector `:scope >` means “direct children of this exact track.” It avoids accidentally selecting cards buried inside some other nested element.

> **Think about it**
> The footer heart has the Font Awesome class `fa-regular fa-heart`, but it is outside this track and does not have `MenueCoffee-Heart`. The menu selector therefore leaves the footer alone.

Why divide by two?

The current track has two matching sets:

```
FIRST SET:   1  2  3  4  5  ...  15
SECOND SET:  1  2  3  4  5  ...  15

  30 cards / 2 = 15 cards per set
```

```
const CoffeeSetSize = CoffeeCards.length / 2;
```

This works only when the track contains exactly two identical sets. If one card is missing from the duplicate, or an extra card is inserted into only one side, the halfway index no longer points to the equivalent card.

What is offsetLeft?

Technical word: `offsetLeft`. In simple words, it is the element's left position relative to its offset parent. In this layout, the cards share the track as their positioning context, so comparing two card positions gives the distance between them.

```
          track start = 0px
                  |
  ├── first card: offsetLeft 0px
  │       gap + card width + gap ...
  └── first duplicate: offsetLeft 4,320px

  loopPoint = 4,320 - 0 = 4,320px
```

```
function GetCoffeeLoopPoint(){
  return CoffeeCards[CoffeeSetSize].offsetLeft
       - CoffeeCards[0].offsetLeft;
}
```

> **Why not hard-code 500 or 50%?**
> Cards use responsive `clamp()` widths and the track uses a gap. The actual distance changes with viewport size. Measuring the rendered positions lets the browser tell us the truth.

Interaction is a hand-off

The automatic carousel and the user should not steer at the same time. The pause flag is a traffic light.

```
GREEN: CoffeeMenuePaused = false  → automatic movement allowed
 RED:   CoffeeMenuePaused = true   → automatic movement waits
```

```
function PauseCoffeeMenue(){
  CoffeeMenuePaused = true;
  clearTimeout(CoffeeMenueResumeTimer);
}

function ResumeCoffeeMenue(){
  clearTimeout(CoffeeMenueResumeTimer);
  CoffeeMenueResumeTimer = setTimeout(() => {
    CoffeeMenuePaused = false;
  }, 1200);
}
```

Calling `clearTimeout` first is important. Scroll events can arrive many times. Clearing the old timer means we keep one clean “resume later” plan rather than letting several old timers wake the carousel unpredictably.

```
USER SCROLLS → PAUSE → USER STOPS → wait 1.2 seconds → RESUME
```

> **Why not resume immediately?**
> A touch release or pointer release does not always mean the user has finished momentum scrolling. The short delay gives the interaction room to settle.

What does “debounce” mean?

Debouncing means waiting until activity has stopped for a chosen amount of time. Each new interaction clears the old timer and starts the waiting period again.

HandleCoffeeMenueScroll()

```
function HandleCoffeeMenueScroll(){
  if(CoffeeMenueWritingScroll)return;

  PauseCoffeeMenue();
  ResumeCoffeeMenue();

  const loopPoint = GetCoffeeLoopPoint();
  if(CoffeeMenueContainer.scrollLeft >= loopPoint){
    CoffeeMenueContainer.scrollLeft -= loopPoint;
  }
}
```

Step	Reason
Ignore guarded events	Do not mistake JavaScript's own movement for a human scroll.
Pause	Give the user control as soon as native scrolling is reported.
Schedule resume	Restart only after 1.2 seconds of quiet.
Read the loop point	Use the current responsive layout.
Subtract one set	Move from the duplicate set to the equivalent first-set position.

Why the writing flag exists

```
JavaScript changes scrollLeft
            ↓
Browser fires scroll event
            ↓
Handler might think the USER scrolled
            ↓
Wrong pause/reset logic

Writing flag = a temporary “this was automatic” label
```

During automatic movement, the code sets `CoffeeMenueWritingScroll = true`. The scroll handler sees it and returns. On the next animation frame, the flag returns to `false`.

> **Possible improvement**
> The exact flag timing is subtle: browser scroll events may be delivered on a different schedule. A production version could use a more explicit internal-scroll guard around the write and reset it after the related scroll event, or compare the latest input event. The current flag teaches the right idea: separate programmatic movement from user movement.

requestAnimationFrame

```
requestAnimationFrame(MoveCoffeeMenue);
```

This asks the browser to call the function before a future repaint. The function moves a little, then schedules itself again:

```
Frame 1 → Frame 2 → Frame 3 → Frame 4 → ...
```

It is a good fit for visual movement because the browser coordinates the callback with painting. It does not promise a perfect 60 FPS; the timing can change when the tab is hidden, the device is busy, or the display has another refresh rate.

Why performance.now() matters

```
const elapsedTime = currentTime - CoffeeMenueLastTime;
CoffeeMenueLastTime = currentTime;
```

```
Frame A: time = 1000ms
Frame B: time = 1016ms
  elapsedTime = 16ms
```

If every frame moved a fixed amount, a slow frame could make the carousel move too slowly or too quickly. Time-based movement says: “move according to how much time passed.”

Speed control

```
CoffeeMenueContainer.scrollLeft += elapsedTime * 0.025;
```

`elapsedTime × 0.025` is the number of pixels to add for that frame. Roughly: `0.01` is slower, `0.025` is the current speed, and `0.05` is faster. These are JavaScript movement factors, not CSS duration units.

Which event means what?

Event	Meaning	Role here
`scroll`	The scroll position changed.	Pause, debounce resume, and check the loop boundary.
`wheel`	Mouse wheel or trackpad input.	Pause automatic movement while the user steers.
`touchstart`	A finger touched the area.	Pause immediately.
`touchend`	The finger left the area.	Schedule resume.
`pointerdown`	A pointer pressed down.	Pause mouse, pen, or compatible touch interaction.
`pointerup`	The pointer was released.	Schedule resume.
`pointercancel`	The browser cancelled the pointer interaction.	Do not leave the carousel stuck in a paused state.

Some events overlap. That is useful because different devices report interaction differently. The scroll handler remains the place that notices the actual position change.

Why passive listeners?

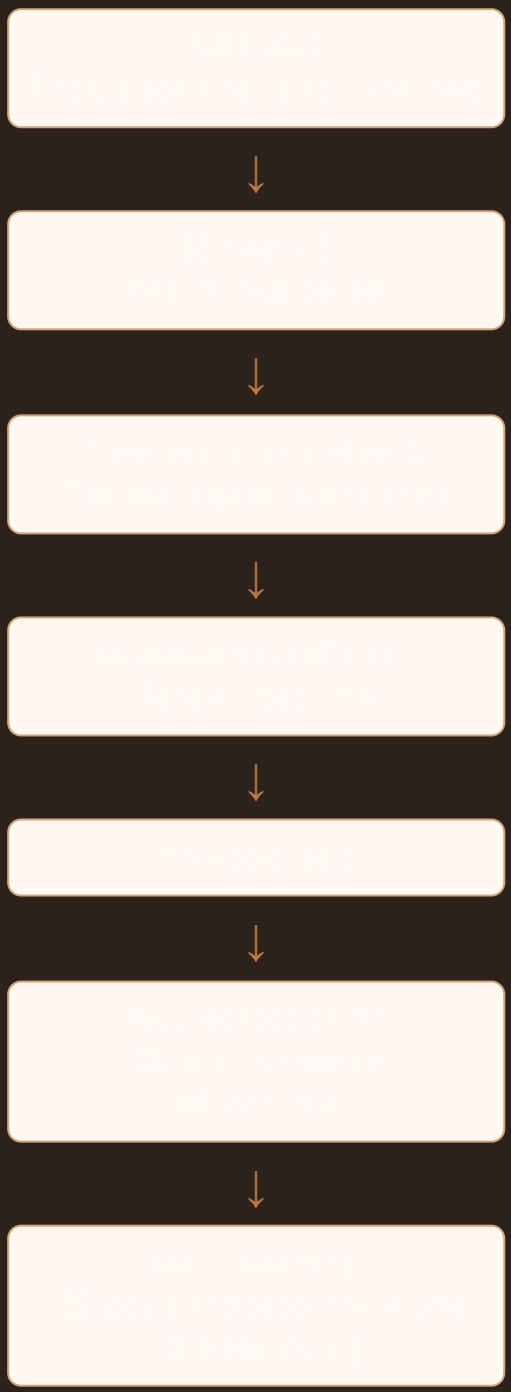
```
addEventListener("scroll", HandleCoffeeMenueScroll, { passive: true });
```

`passive: true` tells the browser that this listener will not call `preventDefault()` to cancel the native action. That is commonly appropriate for scroll/touch observation. It does not magically guarantee faster scrolling; it simply gives the browser a clear constraint.

> **Card behavior stays intact**
> The card's inline navigation handlers are unchanged. Menu hearts still use `stopPropagation()` and `preventDefault()`, so clicking a heart does not navigate to the coffee page. The carousel code does not replace those handlers.

CHAPTER 10

The complete flow



Build it from zero

Stage 1

Make a scroll window

```
.CoffeeMenue-Container {
  overflow-x: auto;
}
```

First prove the browser can scroll.

Stage 2

Add one row

```
.coffee-menue {
  display: flex;
  width: max-content;
  flex-shrink: 0;
}
```

Cards now stay in one wide line.

Stage 3

Duplicate the set

```
A B C D E A B C D E
```

Copy the exact order and design.

Stage 4

Read the DOM

```
const cards = track.querySelectorAll(
  ":scope > .card"
);
```

Count direct card children.

Stages 5-7: measure, move, reset

```
const setSize = cards.length / 2;
const loopPoint = cards[setSize].offsetLeft - cards[0].offsetLeft;
container.scrollLeft += 1;
if (container.scrollLeft >= loopPoint) {
  container.scrollLeft -= loopPoint;
}
```

Do not guess the loop distance. Measure it from the rendered cards.

Stages 8-10: control the loop

```
// Stage 8: pause state
let paused = false;
function pause(){ paused = true; }

// Stage 9: debounce resume
let timer;
function resumeLater(){
  clearTimeout(timer);
  timer = setTimeout(() => paused = false, 1200);
}

// Stage 10: frame loop
function move(){
  if (!paused) container.scrollLeft += 1;
  requestAnimationFrame(move);
}
requestAnimationFrame(move);
```

Build in layers
Do not add every feature at once. First make native scrolling work. Then duplicate. Then measure. Then auto-move. Then pause/resume. After each stage, test one behavior.

A careful recreation checklist

1. Can you drag or wheel the container without JavaScript?
2. Do the two sets look identical?
3. Does the measured loop point land before the second set's end?
4. Does automatic movement stop when you touch or scroll?
5. Does the loop reset to the equivalent visual card?
6. Do hearts still stop card navigation?

What if I remove this?

Removed line	Likely result	What it teaches
`Paused = true`	Autoplay keeps fighting the user's scroll.	Interaction needs ownership.
`clearTimeout(timer)`	Several old timers may resume at unexpected times.	Debounce means one current plan.
`WritingScroll = true`	Automatic movement may look like user input and pause itself.	Programmatic events can trigger browser events too.
`requestAnimationFrame(move)`	The function runs once, then stops.	A frame loop must schedule its next frame.
`scrollLeft -= loopPoint`	The carousel reaches the end of the duplicate and stops or shows the end.	The reset creates the illusion of infinity.
`length / 2`	The wrong card is used as the duplicate-set start.	The markup must contain exactly two matching sets.

Debugging lesson

Removing a line temporarily is an experiment. Observe one behavior, write down what changed, then restore the line. That is more useful than memorizing code.

How carousel bugs usually happen

Mistake	Symptom	Fix
Transform without duplicate content	Blank space or a hard stop.	Use two identical sets and native scroll position.
Hard-code `scrollLeft = 500`	Works on one screen size only.	Measure the first duplicate card with `offsetLeft`.
Use a wrong loop distance	Jump happens too early or too late.	Confirm card count, order, gap, and set boundary.
Start several intervals/RAF loops	Movement becomes too fast or inconsistent.	Create one long-lived frame loop.
Forget to pause	Carousel fights touch or trackpad input.	Pause on interaction and debounce resume.
Ignore scroll events from JavaScript	Autoplay is mistaken for user scrolling.	Use a guarded internal-write state.

Debug with DevTools

```
console.log(CoffeeMenueContainer.scrollLeft);
console.log(CoffeeMenueContainer.scrollWidth);
console.log(CoffeeCards.length);
console.log(GetCoffeeLoopPoint());
```

- `scrollLeft`: current horizontal position.
- `scrollWidth`: total inside width, including content outside the window.
- `CoffeeCards.length`: should be 30 in this project.
- Loop point: should equal the distance from the first card to card 16.

Inspect visually
In Elements, select the outer container. Watch its scroll position while dragging. Select the track and confirm it is one row. Count the card elements directly under the track.

Try before reading the solutions

1. Slow it down. Change the movement factor so the carousel moves more gently.

2. Speed it up. Choose a larger factor and observe how it feels.

3. Change the delay. Make resume wait 2 seconds instead of 1.2.

4. Inspect the loop. Log `GetCoffeeLoopPoint()` and explain what it measures.

5. Remove the reset temporarily. What happens after the second set?

6. Make a tiny carousel. Build one with three cards and a duplicate set.

7. Rebuild without looking. Write the structure, measurement, loop, and pause system from memory, then compare.

Hints

For Exercise 1, change only the speed factor. For Exercise 3, change only `1200`. For Exercise 6, remember that the loop point is not “half the screen”; it is the distance between equivalent cards.

Exercise solutions

1. `elapsedTime * 0.01` is slower than the current `0.025`.
2. `elapsedTime * 0.05` is faster than the current factor.
3. Replace `1200` with `2000`. The unit is milliseconds.
4. Log the function result. It should match the rendered distance from card 1 to card 16, including card widths and gaps.
5. Without the reset, the scroll reaches the end of the duplicated content. There is no third set, so the illusion ends.
6. Use six cards total, `cards.length / 2` gives three, and measure `cards[3].offsetLeft - cards[0].offsetLeft`.
7. Build in the ten stages from this guide. Test each layer before adding the next.

> **Do not copy blindly**
> The important skill is being able to explain why the set size, measured loop point, pause flag, timer, and frame loop all exist. If you can explain the failure caused by removing each one, you understand the architecture.

Possible improvements

Possible Improvement: resize handling
Card widths use responsive CSS. If the viewport changes while the carousel is running, the measured loop distance can change. Re-measuring on resize, and possibly normalizing `scrollLeft`, would make the behavior more robust.

Possible Improvement: reduced motion
Some users prefer less movement. A production carousel could check `prefers-reduced-motion` and disable autoplay while still allowing manual scrolling.

Possible Improvement: structural duplication
The current project manually writes the 15-card set twice. That preserves the existing card handlers, but it also means future card edits must be made twice. A data-driven renderer could generate both sets from one coffee list, provided it keeps heart and navigation behavior correct.

Possible Improvement: interaction timing
The provided code pauses on wheel, touch, and pointer input. The scroll handler also schedules resume. Testing on real trackpads and touch devices is important because momentum scrolling and event timing vary.

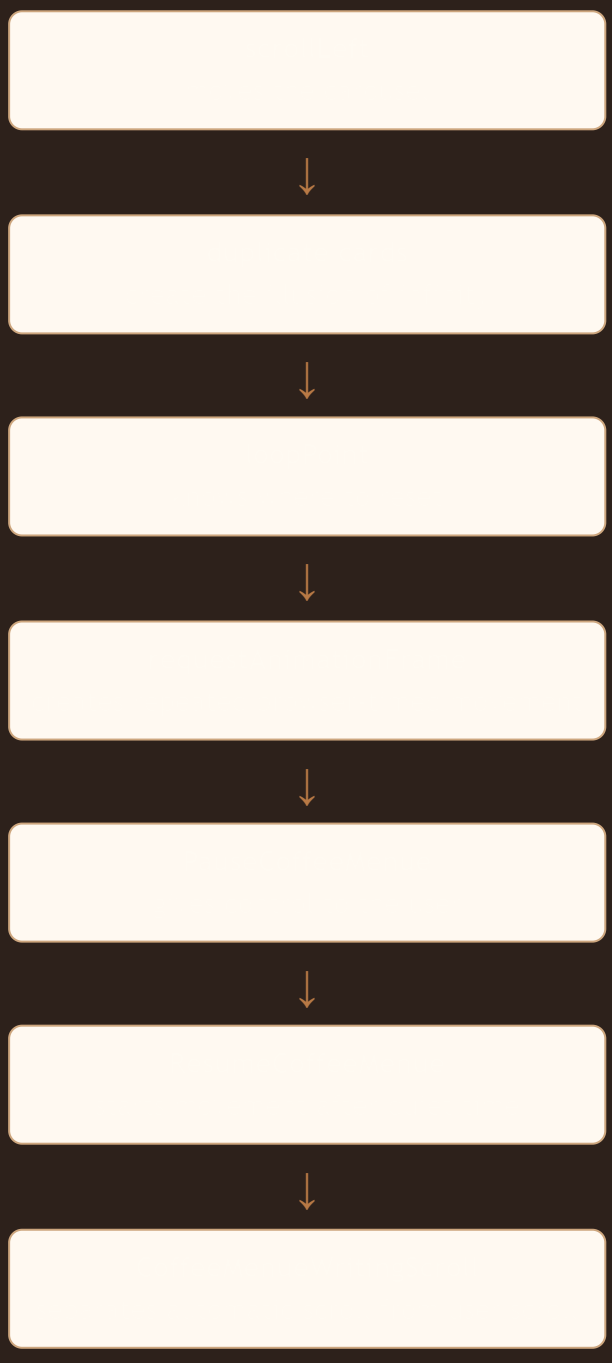
What must not change accidentally?

- The one outer scroll container.
- The one horizontal track containing original plus duplicate cards.
- The menu heart class `MenueCoffee-Heart`.
- The card navigation handlers and heart propagation prevention.

FINAL MENTAL MODEL

Two rows pretending to be infinity

The carousel is not really infinite. It is two identical rows pretending to be one infinite row.



If you remember one sentence:

Move the real scroll position, measure the distance to the duplicate, and when you reach it, subtract one complete set so the same picture stays under the user's eyes.